

Rozwiązywanie problemu optymalizacyjnego w grze sudoku.

Gracjan Kościński

Styczeń 2026

Spis treści

1	Wstęp	2
2	Algorytm backtrackingu	2
2.1	Zasada działania oraz wyniki	2
3	Algorytm genetyczny z wykorzystaniem biblioteki pygad	4
3.1	Populacja początkowa	4
3.2	Funkcja fitness	5
3.3	Lokalne przeszukiwanie (Local Search)	5
3.4	Krzyżowanie (Crossover)	7
3.5	Mutacje	7
3.6	Mechanizm kontroli i migracji (Island Model)	8
3.7	Wyniki	10
4	Algorytm ACO	11
4.1	Inicjalizacja i strategia MRV	11
4.2	Wyznaczanie kandydatów	12
4.3	Funkcja fitness	12
4.4	Konfiguracja i uruchomienie algorytmu	13
4.5	Wyniki	15
5	Algorytm PSO	17
5.1	Inicjalizacja i analiza struktury problemu	17
5.2	Rekonstrukcja siatki	18
5.3	Funkcja fitness	19
5.4	Konfiguracja i uruchomienie algorytmu	20
5.5	Wyniki	21
6	Podsumowanie	23

1 Wstęp

Sudoku to łamigłówka logiczna, która polega na wypełnieniu 9x9 siatki cyframi od 1 do 9 w taki sposób, aby każda cyfra pojawiała się tylko raz w każdym wierszu, kolumnie oraz w każdej z dziewięciu 3x3 podsiatek (nazywanych także „blokami” lub „regionami”). W poniższej pracy przedstawię porównanie algorytmów rozwiązujących problem gry w sudoku (Backtracking, algorytm genetyczny oraz PSO) dla różnego stopnia skomplikowania gry.

2 Algorytm backtrackingu

2.1 Zasada działania oraz wyniki

Algorytm rozwiązywania sudoku za pomocą backtrackingu jest klasycznym przykładem zastosowania rekurencji w rozwiązywaniu problemów kombinatorycznych.

```
def is_valid(board, row, col, num):
    for i in range(9):
        if board[row][i] == num or board[i][col] == num:
            return False

    start_row, start_col = 3 * (row // 3), 3 * (col // 3)
    for i in range(3):
        for j in range(3):
            if board[start_row + i][start_col + j] == num:
                return False

    return True

def solve_sudoku(board):
    empty = find_empty(board)
    if not empty:
        return True

    row, col = empty
    for num in range(1, 10):
        if is_valid(board, row, col, num):
            board[row][col] = num

            if solve_sudoku(board):
                return True

            board[row][col] = 0

    return False

def find_empty(board):
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                return (i, j)

    return None
```

Algorytm szuka pierwszego pustego pola za pomocą `find_empty`. Wybiera pierwsze puste pole i próbuje wstawić kolejno liczby od 1 do 9. Sprawdzając każdą liczbę za pomocą funkcji `is_valid`, aby upewnić się, że nie łamie reguł Sudoku (czy liczba ta nie powtarza się w tej samej kolumnie, w tym samym rzędzie ani w tym samym 3x3 podkwadracie). Jeśli znajdzie liczbę, która pasuje, wstawia ją i rekurencyjnie próbuje rozwiązać resztę planszy. Jeśli rozwiązanie jest niemożliwe dla danej liczby, resetuje pole i wraca (backtracking) do poprzedniego kroku. Wynik algorytmu dla prostego problemu (40/81 początkowo zapełnionych pól) daje 100% poprawnych odpowiedzi, co pokazane jest na rysunku 1. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 0,020 s.

Initial Sudoku Puzzle									Computed Solution								
9		2		7	8	4			9	6	2	3	7	8	4	1	5
1	8	5				7	6		1	8	5	4	2	9	7	6	3
	7		5						3	7	4	5	6	1	9	2	8
7	5				6		8	4	7	5	3	1	9	6	2	8	4
4		6				1	5	7	4	9	6	8	3	2	1	5	7
	1			4	5	3	9	6	2	1	8	7	4	5	3	9	6
	2					5		9	8	2	7	6	1	3	5	4	9
6		9			7	8	3		6	4	9	2	5	7	8	3	1
			9	8	4		7		5	3	1	9	8	4	6	7	2

Rysunek 1: Wynik działania algorytmu brute-force dla prostego problemu.

Następnie sprawdzono działanie algorytmu na bardziej wymagającym problemie (38/81 początkowo zapełnionych pól). Wyniki widoczne są na rysunku 2. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 0,033 s.

Initial Sudoku Puzzle									Computed Solution								
9	6					4			9	6	2	3	7	8	4	1	5
1	8	5	4	2					1	8	5	4	2	9	7	6	3
					1	9		8	3	7	4	5	6	1	9	2	8
5	3		9	8		6			5	3	1	9	8	4	6	7	2
	4	9						1	6	4	9	2	5	7	8	3	1
8	2	7					4	9	8	2	7	6	1	3	5	4	9
7	5		1		6		8		7	5	3	1	9	6	2	8	4
		6	8	3			5		4	9	6	8	3	2	1	5	7
	1	8	7			3		6	2	1	8	7	4	5	3	9	6

Rysunek 2: Wynik działania algorytmu brute-force dla problemu średniej trudności.

Sprawdzono również działanie algorytmu na jeszcze bardziej wymagającym problemie (32/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 3. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 0,041s.

Initial Sudoku Puzzle									Computed Solution								
					1	2	3		6	5	7	9	4	1	2	3	8
1	2	3			8		4		1	2	3	6	5	8	9	4	7
8		4			7	6	5		8	9	4	2	3	7	6	5	1
7	6	5							7	6	5	1	2	3	4	8	9
									2	3	1	8	9	4	5	7	6
						1	2	3	9	4	8	7	6	5	1	2	3
	1	2	3			8		4	5	1	2	3	7	6	8	9	4
	8		4			7	6	5	3	8	9	4	1	2	7	6	5
	7	6	5						4	7	6	5	8	9	3	1	2

Rysunek 3: Wynik działania algorytmu brute-force dla problemu ciężkiego.

3 Algorytm genetyczny z wykorzystaniem biblioteki pygad

Zadaniem tego algorytmu genetycznego jest znalezienie jak najlepszego rozwiązania sudoku poprzez iteracyjne ulepszanie populacji rozwiązań przy użyciu operacji selekcji, krzyżowania i mutacji. W porównaniu do wcześniejszej prostej wersji, obecna implementacja zawiera znaczące usprawnienia, w tym lokalne przeszukiwanie (local search) oraz mechanizm migracji oparty na modelu wyspowym (Island Model).

Początkowa populacja to domyślnie 1000 osobników, liczba generacji to 1000. W implementacji algorytmu genetycznego do rozwiązywania sudoku użyto metody selekcji rodziców o nazwie **Tournament Selection** z parametrem $K=5$.

Metoda **Tournament Selection** działa w następujący sposób:

- **Losowanie Grupy Kandydatów:** Z populacji losowo wybiera się K osobników (w tym przypadku $K=5$), którzy uczestniczą w turnieju.
- **Wybór Najlepszego:** Spośród wylosowanych osobników, ten z najwyższą wartością funkcji fitness zostaje wybrany jako rodzic.
- **Powtarzanie Procesu:** Proces ten powtarza się wielokrotnie, aby uzyskać wymaganą liczbę rodziców do krzyżowania.

3.1 Populacja początkowa

Populacja początkowa jest generowana przy użyciu funkcji `create_initial_population(size)`. Każdy osobnik w populacji początkowej jest generowany poprzez wypełnienie pustych miejsc w siatce sudoku liczbami od 1 do 9, które nie powtarzają się w swoich 3×3 podsiatkach. Takie podejście zapewnia dobry start, dzięki któremu każda 3×3 podsiatka zawiera wszystkie cyfry od 1 do 9, co znacząco redukuje przestrzeń poszukiwań.

```

def create_initial_population(self, size):
    population = []
    for _ in range(size):
        individual = self.initial_sudoku.copy()
        for i in range(0, 9, 3):
            for j in range(0, 9, 3):
                subgrid = individual[i : i + 3, j : j + 3].flatten()
                missing = [num for num in range(1, 10)
                           if num not in subgrid]
                np.random.shuffle(missing)
                subgrid[subgrid == 0] = missing
                individual[i : i + 3, j : j + 3] = subgrid.reshape((3, 3))
        population.append(individual.flatten())
    return np.array(population)

```

3.2 Funkcja fitness

Funkcja fitness (`fitness_func`) ocenia jakość każdego rozwiązania sudoku w populacji. W przeciwieństwie do poprzedniej wersji, która penalizowała powtórzenia, obecna implementacja bezpośrednio nagradza unikalność:

- **Unikalne liczby w wierszach:** Dla każdego wiersza zlicza się liczbę unikalnych cyfr (maksymalnie 9).
- **Unikalne liczby w kolumnach:** Dla każdej kolumny zlicza się liczbę unikalnych cyfr (maksymalnie 9).

Maksymalna wartość fitness wynosi 162 punktów ($9 \text{ wierszy} \times 9 \text{ unikalnych cyfr} + 9 \text{ kolumn} \times 9 \text{ unikalnych cyfr}$), co odpowiada prawidłowo rozwiązanemu sudoku. Ponieważ populacja początkowa zapewnia już poprawne 3x3 podsiatki, funkcja fitness koncentruje się wyłącznie na wierszach i kolumnach.

```

def fitness_func(self, ga_instance, solution, solution_idx):
    """Fitness: suma unikalnych cyfr w rzędach i kolumnach (Max: 162)."""
    grid = solution.reshape((9, 9)).astype(int)
    score = 0
    for i in range(9):
        score += len(np.unique(grid[i, :])) # Unikalne w wierszu
        score += len(np.unique(grid[:, i])) # Unikalne w kolumnie
    return float(score)

```

3.3 Lokalne przeszukiwanie (Local Search)

Kluczową innowacją w obecnej implementacji jest mechanizm lokalnego przeszukiwania oparty na metodzie Iterative Hill Climbing. Funkcja `local_search` naprawia błędy w rozwiązaniu poprzez iteracyjne zamiany wartości w obrębie 3x3 podsiatek, aż do osiągnięcia lokalnego maksimum fitness.

Algorytm działa następująco: W każdej iteracji bloki 3x3 są przeglądane w losowej kolejności, co zwiększa różnorodność przeszukiwania. Dla każdej pary modyfikowalnych pozycji w bloku testowana jest zamiana wartości. Jeśli zamiana prowadzi do wzrostu fitness, jest natychmiast akceptowana. Proces trwa do momentu, gdy żadna zamiana nie poprawia fitness. Lokalne przeszukiwanie jest uruchamiane dla najlepszego osobnika w generacji, gdy jego fitness przekracza próg 150 punktów.

```

def local_search(self, individual):
    """Iterative Hill Climbing"""
    grid = individual.reshape((9, 9)).astype(int)
    best_fitness = self.fitness_func(None, individual, None)

    while True:
        improved = False
        blocks = np.arange(9)
        np.random.shuffle(blocks)

        for block_id in blocks:
            r, c = (block_id // 3) * 3, (block_id % 3) * 3
            fixed_block = self.fixed_positions[r : r + 3,
                                                c : c + 3].flatten()

            mutable_indices = np.where(~fixed_block)[0]
            if len(mutable_indices) < 2:
                continue

            found_better = False
            for i in range(len(mutable_indices)):
                for j in range(i + 1, len(mutable_indices)):
                    idx1, idx2 = mutable_indices[i], mutable_indices[j]
                    flat_sub = grid[r : r + 3, c : c + 3].flatten()
                    # Swap
                    flat_sub[idx1], flat_sub[idx2] =
                    flat_sub[idx2], flat_sub[idx1]

                    test_grid = grid.copy()
                    test_grid[r : r + 3, c : c + 3] =
                    flat_sub.reshape(3, 3)

                    new_fitness = self.fitness_func(None,
                                                    test_grid.flatten(), None)
                    if new_fitness > best_fitness:
                        grid = test_grid
                        best_fitness = new_fitness
                        improved = True
                        found_better = True
                        break
                if found_better:
                    break
            if found_better:
                break
        if not improved:
            break
    return grid.flatten()

```

3.4 Krzyżowanie (Crossover)

Zaimplementowano niestandardowe krzyżowanie (`custom_crossover`), które działa na zasadzie krzyżowania jednopunktowego na poziomie bloków 3x3. Dla każdego z 9 bloków losowo decyduje się, czy zostanie odziedziczony od pierwszego czy drugiego rodzica, co pozwala zachować integralność podsiatek.

```
def custom_crossover(self, parents, offspring_size, ga_instance):
    offspring = []
    for k in range(offspring_size[0]):
        p1 = parents[k % parents.shape[0]].reshape(9, 9)
        p2 = parents[(k + 1) % parents.shape[0]].reshape(9, 9)
        child = np.copy(p1)

        # Crossover na poziomie bloków 3x3
        for b in range(9):
            if np.random.rand() < 0.5:
                r, c = (b // 3) * 3, (b % 3) * 3
                child[r : r + 3, c : c + 3] = p2[r : r + 3, c : c + 3]
        offspring.append(child.flatten())
    return np.array(offspring)
```

3.5 Mutacje

Mutacja (`custom_mutation`) została znacząco ulepszona poprzez wprowadzenie mechanizmu adaptacyjnego. Prawdopodobieństwo mutacji dynamicznie dostosowuje się do stanu algorytmu:

- **Normalna mutacja (10%):** W przypadku regularnego postępu algorytmu stosowane jest niskie prawdopodobieństwo mutacji (0.1).
- **Intensywna mutacja (40%):** Gdy wykryta zostanie stagnacja (brak poprawy przez 50+ generacji), prawdopodobieństwo mutacji wzrasta do 0.4, co zwiększa eksplorację przestrzeni rozwiązań.
- **Zamiana w obrębie bloku:** Mutacja polega na losowej zamianie dwóch pozycji w losowo wybranym bloku 3x3, przy zachowaniu stałych pozycji.

```
def custom_mutation(self, offspring, ga_instance):
    # Adaptacyjna szansa na mutację
    rate = 0.4 if self.stagnation_counter > 50 else 0.1

    for idx in range(offspring.shape[0]):
        if np.random.rand() < rate:
            grid = offspring[idx].reshape(9, 9)
            b = np.random.randint(0, 9)
            r, c = (b // 3) * 3, (b % 3) * 3
            fixed_sub = self.fixed_positions[r : r + 3,
                                             c : c + 3].flatten()
            m_idx = np.where(~fixed_sub)[0]
            if len(m_idx) >= 2:
                i1, i2 = np.random.choice(m_idx, 2, replace=False)
                flat = grid[r : r + 3, c : c + 3].flatten()
                flat[i1], flat[i2] = flat[i2], flat[i1]
                grid[r : r + 3, c : c + 3] = flat.reshape(3, 3)
            offspring[idx] = grid.flatten()
    return offspring
```

3.6 Mechanizm kontroli i migracji (Island Model)

Funkcja `on_generation` monitoruje postęp algorytmu w każdej generacji i implementuje zaawansowane mechanizmy optymalizacyjne:

- **Wywołanie lokalnego przeszukiwania:** Dla najlepszego osobnika z $\text{fitness} \geq 150$ automatycznie uruchamiane jest lokalne przeszukiwanie.
- **Detekcja stagnacji:** Algorytm śledzi liczbę kolejnych generacji bez poprawy fitness (`stagnation_counter`).
- **Interwencja przy stagnacji:** Gdy fitness jest bardzo wysoki (≥ 158) ale stabilny przez 15+ generacji, najlepszy osobnik poddawany jest dodatkowej mutacji w celu wyjścia z lokalnego optimum.
- **Model wyspowy (Island Model):** Po 200 generacjach stagnacji następuje drastyczna migracja - 80% najsłabszych osobników w populacji jest zastępowanych nowymi, losowo wygenerowanymi rozwiązaniami. Ten mechanizm symuluje migrację między izolowanymi populacjami i jest szczególnie skuteczny w przełamywaniu długotrwałych stagnacji.
- **Warunek stopu:** Algorytm zatrzymuje się automatycznie po osiągnięciu $\text{fitness} = 162$ (pełne rozwiązanie).


```

def on_generation(self, ga_instance):
    best_sol, best_fit, best_idx = ga_instance.best_solution()

    # Lokalne przeszukiwanie dla obiecujących rozwiązań
    if best_fit >= 150:
        improved = self.local_search(best_sol)
        improved_fit = self.fitness_func(None, improved, None)

        # Dodatkowa mutacja przy wysokim fitness i stagnacji
        if (improved_fit == best_fit and
            self.stagnation_counter > 15 and
            best_fit >= 158):
            improved = self.custom_mutation(
                improved.reshape(1, -1), ga_instance)[0]
            improved_fit = self.fitness_func(None, improved, None)
            self.stagnation_counter = 0

        ga_instance.population[best_idx] = improved
        best_sol = improved
        best_fit = improved_fit

    self.best_fitness_history.append(best_fit)

    # Raportowanie postępu co 100 generacji
    if ga_instance.generations_completed % 100 == 0:
        print(f"Gen {ga_instance.generations_completed}: "
              f"Fitness {best_fit:.0f}/162 "
              f"(Stagnacja: {self.stagnation_counter})")

    # Warunek stopu - rozwiązanie znalezione
    if best_fit == 162:
        print(f"!!! Rozwiązano w pokoleniu "
              f"{ga_instance.generations_completed} !!!")
        self.found_solution = best_sol.reshape(9, 9)
        return "stop"

    # Śledzenie stagnacji
    if (len(self.best_fitness_history) > 1 and
        best_fit <= self.best_fitness_history[-2]):
        self.stagnation_counter += 1
    else:
        self.stagnation_counter = 0

    # Migracja przy długiej stagnacji (Island Model)
    if self.stagnation_counter >= self.max_stagnation:
        print(" -> Wykryto stagnację. "
              "Migracja nowych osobników (Island Model)...")
        num_new = int(ga_instance.sol_per_pop * 0.8)
        new_pop = self.create_initial_population(num_new)
        indices = np.argsort(
            ga_instance.last_generation_fitness)[:num_new]
        ga_instance.population[indices] = new_pop
        self.stagnation_counter = 0

```

3.7 Wyniki

Wynik algorytmu dla prostego problemu (40/81 początkowo wypełnionych pól) daje 100% poprawnych odpowiedzi, co pokazane jest na rysunku 4.

Czas działania algorytmu jako średnia z 10 odpaleń wyniósł 36,889 s.

Initial Sudoku Puzzle									Computed Solution								
9		2		7	8	4			9	6	2	3	7	8	4	1	5
1	8	5				7	6		1	8	5	4	2	9	7	6	3
	7		5						3	7	4	5	6	1	9	2	8
7	5				6		8	4	7	5	3	1	9	6	2	8	4
4		6				1	5	7	4	9	6	8	3	2	1	5	7
	1			4	5	3	9	6	2	1	8	7	4	5	3	9	6
	2					5		9	8	2	7	6	1	3	5	4	9
6		9			7	8	3		6	4	9	2	5	7	8	3	1
			9	8	4		7		5	3	1	9	8	4	6	7	2

Rysunek 4: Wynik działania algorytmu genetycznego dla prostego problemu.

Następnie sprawdzono działanie algorytmu na bardziej wymagającym problemie (38/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 5. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 5,041 s.

Initial Sudoku Puzzle									Computed Solution								
9	6					4			9	6	2	3	7	8	4	1	5
1	8	5	4	2					1	8	5	4	2	9	7	6	3
					1	9		8	3	7	4	5	6	1	9	2	8
5	3		9	8		6			5	3	1	9	8	4	6	7	2
	4	9						1	6	4	9	2	5	7	8	3	1
8	2	7					4	9	8	2	7	6	1	3	5	4	9
7	5		1		6		8		7	5	3	1	9	6	2	8	4
		6	8	3			5		4	9	6	8	3	2	1	5	7
	1	8	7			3		6	2	1	8	7	4	5	3	9	6

Rysunek 5: Wynik działania algorytmu genetycznego dla problemu średniej trudności.

Sprawdzono również działanie algorytmu na jeszcze bardziej wymagającym problemie (32/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 6. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 84,250s. W przypadku tak zaawansowanego problemu algorytm często zatrzymuje się w lokalnym minimum i nie potrafi się już poprawić, aż do końca swojego działania. Dla podanego problemu algorytm osiąga dokładność poprawnych odpowiedzi na poziomie około 86,5%.

Initial Sudoku Puzzle									Computed Solution								
					1	2	3		6	5	7	9	4	1	2	3	8
1	2	3			8		4		1	2	3	6	5	8	9	4	7
8		4			7	6	5		8	9	4	2	3	7	6	5	1
7	6	5							7	6	5	1	4	3	5	8	9
									9	3	1	8	2	5	4	7	6
						1	2	3	2	4	8	7	9	6	1	2	3
	1	2	3			8		4	5	1	2	3	6	7	8	9	4
	8		4			7	6	5	3	8	9	4	1	2	7	6	5
	7	6	5						4	7	6	5	8	9	3	1	2

Rysunek 6: Wynik działania algorytmu genetycznego dla problemu ciężkiego.

4 Algorytm ACO

Algorytm ACO (Ant Colony Optimization) to metaheurystyka inspirowana zachowaniem rzeczywistych kolonii mrówek, wykorzystywana do rozwiązywania problemów optymalizacji kombinatorycznej. W kontekście Sudoku algorytm symuluje proces, w którym wirtualne mrówki eksplorują przestrzeń możliwych wypełnień siatki, pozostawiając ślady feromonowe na ścieżkach prowadzących do rozwiązań o najniższej liczbie naruszeń zasad.

W implementacji wykorzystano bibliotekę `mealpy`, która oferuje wariant ACOR (Ant Colony Optimization for Continuous Domains). Został on dostosowany do problemu Sudoku poprzez zdefiniowanie dyskretnych granic zmiennych odpowiadających indeksom na listach dopuszczalnych kandydatów.

4.1 Inicjalizacja i strategia MRV

Klasa `SudokuACO` przygotowuje problem poprzez transformację planszy 9×9 do postaci zrozumiałej dla algorytmu. Kluczowym elementem jest zastosowanie heurystyki **MRV (Minimum Remaining Values)**, która sortuje puste pola według stopnia ich ograniczenia.

```

def init(self, initial_grid): self.initial_grid = initial_grid.copy()
# Znajdź wszystkie puste pola (wartość 0)
# all_empty = list(zip(*np.where(self.initial_grid == 0)))

# Sortuj pola według liczby kandydatów (MRV)
all_empty.sort(
    key=lambda pos: len(self._get_candidates(
        pos[0], pos[1], self.initial_grid))
)

self.empty_pos = all_empty
self.candidates = [
    self._get_candidates(r, c, self.initial_grid) for r, c in self.empty_pos
]
self.n_vars = len(self.empty_pos)

```

Zastosowanie MRV sprawia, że algorytm w pierwszej kolejności wypełnia pola, dla których istnieje najmniej poprawnych opcji. Dzięki temu trudne decyzje są podejmowane na początku, co drastycznie ogranicza liczbę błędnych ścieżek analizowanych przez mrówki.

4.2 Wyznaczanie kandydatów

Dla każdego pustego pola funkcja `_get_candidates` analizuje więzy narzucone przez rzędy, kolumny i bloki 3×3. Pozwala to na redukcję dziedziny zmiennych, co jest specyficznym dostosowaniem ACO do logiki Sudoku.

```

def _get_candidates(self, r, c, grid):
    row_vals = set(grid[r, :])
    col_vals = set(grid[:, c])
    r_start, c_start = (r // 3) * 3, (c // 3) * 3
    square_vals = set(grid[r_start : r_start + 3, c_start : c_start + 3].flatten())

    forbidden = row_vals | col_vals | square_vals
    allowed = [i for i in range(1, 10) if i not in forbidden]

    # Jeśli brak dozwolonych cyfr, zwróć pełny zakres (fallback)
    return allowed if allowed else list(range(1, 10))

```

Zamiast wybierać spośród 9 cyfr, algorytm optymalizuje wybór indeksu z listy `allowed`. To podejście gwarantuje, że mrówki poruszają się jedynie w obszarze potencjalnie poprawnych rozwiązań lokalnych.

4.3 Funkcja fitness

Wartość funkcji celu (penalty) informuje kolonię o jakości znalezionej rozwiązania. Zastosowano tutaj kwadraty różnic, co pozwala na silniejsze różnicowanie wyników i szybszą eliminację błędnych konfiguracji.

```

def fitness_function(self, solution):
    grid = self.initial_grid.copy()
    for idx, cand_idx in enumerate(solution):
        r, c = self.empty_pos[idx]
        cand_list = self.candidates[idx]
        grid[r, c] = cand_list[int(cand_idx) % len(cand_list)]

    penalty = 0
    # Sprawdzanie wierszy i kolumn
    for i in range(9):
        row = grid[i, :]
        col = grid[:, i]
        # Używam (9 - unikalne) * mnożnik, aby mocniej różnicować wyniki
        penalty += (9 - len(np.unique(row))) ** 2
        penalty += (9 - len(np.unique(col))) ** 2

    # Kwadraty 3x3
    r_s, c_s = (i // 3) * 3, (i % 3) * 3
    subgrid = grid[r_s : r_s + 3, c_s : c_s + 3]
    penalty += (9 - len(np.unique(subgrid))) ** 2

    return float(penalty)

```

Wzór ten karze rozwiązanie tym mocniej, im więcej powtórek występuje w danej jednostce (wierszu/kolumnie/bloku). Idealne rozwiązanie osiąga wartość 0.0.

4.4 Konfiguracja i uruchomienie algorytmu

Główna metoda solve konfiguruje i uruchamia algorytm ACOR, transformując problem sudoku w format zrozumiały dla biblioteki mealpy. Proces rozpoczyna się od sprawdzenia trywialnego przypadku - jeśli sudoku jest już rozwiązane (brak pustych pól), funkcja natychmiast zwraca siatkę początkową z fitness 0. W przeciwnym razie przygotowana jest definicja problemu optymalizacji. Dla każdego pustego pola określone są granice zmiennych decyzyjnych - dolna granica (lb) wynosi 0, co odpowiada pierwszemu kandydatowi na liście, a górna granica (ub) to długość listy kandydatów minus 1, co odpowiada ostatniemu kandydatowi.

Te granice tworzą dyskretną przestrzeń poszukiwań, gdzie każda zmienna decyzyjna może przyjąć wartość całkowitą z określonego zakresu. Następnie tworzony jest słownik problem_dict zawierający trzy kluczowe elementy: obiekt IntegerVar definiujący granice i typ zmiennych, specyfikację "minmax" jako "min" (ponieważ minimalizujemy funkcję penalty), oraz referencję do funkcji celu (fitness_function).

Kolejnym krokiem jest utworzenie instancji algorytmu OriginalACOR z biblioteki mealpy, z parametrami epoch=1000 (maksymalna liczba iteracji), pop_size=200 (wielkość kolonii mrówek), oraz sample_count=50 (liczba najlepszych rozwiązań wykorzystywanych do aktualizacji feromonów). Konfiguracja warunku zakończenia (termination) określa, że algorytm powinien zatrzymać się po osiągnięciu wartości docelowej 0.0 (z tolerancją epsilon=1e-6) lub po 150 epokach bez poprawy.

Po skonfigurowaniu wszystkich parametrów wywoływana jest metoda solve, która uruchamia główną pętlę algorytmu. Po zakończeniu obliczeń najlepsze znalezione rozwiązanie jest dekodowane z postaci wektora indeksów na konkretną siatkę sudoku poprzez wypełnienie pustych pól odpowiednimi cyframi z list kandydatów.

```

def solve(self):
    if self.n_vars == 0: # Sudoku już rozwiązane
        return self.initial_grid, 0.0

    # Definicja granic zmiennych (indeksy list kandydatów)
    lb = [0] * self.n_vars
    ub = [len(c) - 1 for c in self.candidates]

    problem_dict = {
        "bounds": IntegerVar(lb=lb, ub=ub),
        "minmax": "min",
        "obj_func": self.fitness_function,
    }

    # Konfiguracja algorytmu ACOR
    model = ACOR.OriginalACOR(
        epoch=1000, # Maksymalna liczba iteracji
        pop_size=200, # Liczba mrówek w kolonii
        sample_count=50 # Liczba próbek do aktualizacji feromonów
    )

    # Warunek zakończenia
    termination = {
        "mode": "target", # Zakończ gdy osiągnięto cel
        "quantity": 0.0, # Cel: fitness = 0
        "epsilon": 1e-6, # Tolerancja numeryczna
        "max_early_stop": 150, # Zatrzymaj po 150 epokach bez poprawy
    }

    g_best = model.solve(problem_dict, termination=termination)

    # Dekodowanie najlepszego rozwiązania
    final_grid = self.initial_grid.copy()
    for idx, cand_idx in enumerate(g_best.solution):
        r, c = self.empty_pos[idx]
        final_grid[r, c] = self.candidates[idx][int(cand_idx)]

    return final_grid, g_best.target

```

4.5 Wyniki

Wynik algorytmu dla prostego problemu (40/81 początkowo wypełnionych pól) daje 100% poprawnych odpowiedzi, co pokazane jest na rysunku 7.

Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 53,412 s.

Initial Sudoku Puzzle									Computed Solution								
9		2		7	8	4			9	6	2	3	7	8	4	1	5
1	8	5				7	6		1	8	5	4	2	9	7	6	3
	7		5						3	7	4	5	6	1	9	2	8
7	5				6		8	4	7	5	3	1	9	6	2	8	4
4		6				1	5	7	4	9	6	8	3	2	1	5	7
	1			4	5	3	9	6	2	1	8	7	4	5	3	9	6
	2					5		9	8	2	7	6	1	3	5	4	9
6		9			7	8	3		6	4	9	2	5	7	8	3	1
			9	8	4		7		5	3	1	9	8	4	6	7	2

Rysunek 7: Wynik działania algorytmu ACO dla prostego problemu.

Następnie sprawdzono działanie algorytmu na bardziej wymagającym problemie (38/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 8. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 67,136 s.

Initial Sudoku Puzzle									Computed Solution								
9	6					4			9	6	2	3	7	8	4	1	5
1	8	5	4	2					1	8	5	4	2	9	7	6	3
					1	9		8	3	7	4	5	6	1	9	2	8
5	3		9	8		6			5	3	1	9	8	4	6	7	2
	4	9						1	6	4	9	2	5	7	8	3	1
8	2	7					4	9	8	2	7	6	1	3	5	4	9
7	5		1		6		8		7	5	3	1	9	6	2	8	4
		6	8	3			5		4	9	6	8	3	2	1	5	7
	1	8	7			3		6	2	1	8	7	4	5	3	9	6

Rysunek 8: Wynik działania algorytmu ACO dla problemu średniej trudności.

Sprawdzono również działanie algorytmu na jeszcze bardziej wymagającym problemie (32/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 9. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 42,151s. Algorytm osiągnął średni wynik na poziomie 89%

Initial Sudoku Puzzle									Computed Solution								
					1	2	3		6	5	7	6	4	1	2	3	9
1	2	3			8		4		1	2	3	9	5	8	9	4	7
8		4			7	6	5		8	9	4	2	3	7	6	5	1
7	6	5							7	6	5	1	2	3	4	8	8
									2	3	1	2	9	4	5	7	6
						1	2	3	9	4	8	6	5	4	1	2	3
	1	2	3			8		4	5	1	2	3	7	6	8	9	4
	8		4			7	6	5	3	8	9	4	1	2	7	6	5
	7	6	5						4	7	6	5	8	2	3	1	2

Rysunek 9: Wynik działania algorytmu ACO dla problemu ciężkiego.

5 Algorytm PSO

Algorytm PSO (Particle Swarm Optimization) to metaheurystyka inspirowana zachowaniem społecznym ptaków lub ryb poruszających się w stadzie, używana do rozwiązywania problemów optymalizacji ciągłej i dyskretnej. W kontekście sudoku, algorytm symuluje rój cząstek (particles), z których każda reprezentuje potencjalne rozwiązanie problemu, poruszających się w przestrzeni poszukiwań w kierunku coraz lepszych rozwiązań, kierując się zarówno własnym doświadczeniem, jak i sukcesami innych cząstek w roju.

W implementacji wykorzystano bibliotekę pyswarms, która jest aktywnie rozwijaną i popularną implementacją algorytmu PSO w Pythonie, oferującą zarówno podstawowe, jak i zaawansowane warianty algorytmu. Algorytm operuje na roju składającym się z 200 cząstek przez maksymalnie 2000 iteracji. Kluczową innowacją w tej implementacji jest zastosowanie mechanizmu sortowania (argsort) do przekształcania ciągłych pozycji cząstek na dyskretne permutacje cyfr, co naturalnie zapewnia unikalność cyfr w blokach 3x3 bez konieczności dodatkowych mechanizmów naprawczych.

Jak działa PSO? Proces rozpoczyna się od inicjalizacji roju cząstek, gdzie każda cząstka otrzymuje losową pozycję w przestrzeni poszukiwań oraz losową prędkość. Pozycja każdej cząstki koduje potencjalne rozwiązanie sudoku - dla każdego bloku 3x3 cząstka przechowuje zestaw wartości ciągłych (wag), które następnie są przekształcane przez sortowanie na permutację brakujących cyfr w tym bloku. W każdej iteracji algorytmu każda cząstka oceniana jest przez funkcję fitness, która oblicza jakość reprezentowanego przez nią rozwiązania.

Następnie dla każdej cząstki aktualizowana jest informacja o jej najlepszej dotychczas osiągniętej pozycji (personal best), a także identyfikowana jest najlepsza pozycja znaleziona przez cały rój (global best). Na podstawie tych informacji obliczana jest nowa prędkość cząstki, która składa się z trzech komponentów: bezwładności (tendencja do kontynuowania obecnego kierunku ruchu, kontrolowana przez parametr $w=0.8$), przyciągania do własnego najlepszego znalezionego rozwiązania (komponent kognitywny, kontrolowany przez $c1=1.4$), oraz przyciągania do najlepszego rozwiązania znalezionego przez rój (komponent społeczny, kontrolowany przez $c2=0.7$). Po obliczeniu nowej prędkości aktualizowana jest pozycja cząstki poprzez dodanie prędkości do obecnej pozycji. Cały proces powtarza się przez ustaloną liczbę iteracji, przy czym rój stopniowo zbliżony w kierunku obszarów przestrzeni zawierających lepsze rozwiązania.

5.1 Inicjalizacja i analiza struktury problemu

Klasa SudokuPSOSolver rozpoczyna pracę od szczegółowej analizy struktury początkowej planszy sudoku, identyfikując dla każdego z dziewięciu bloków 3x3 zarówno cyfry, które już są obecne, jak i te, których brakuje i muszą zostać uzupełnione. Ten proces jest kluczowy, ponieważ redukuje problem z wypełniania 81 pól (potencjalnie 9^{81} kombinacji) do znacznie mniejszego problemu ustalania permutacji brakujących cyfr w każdym bloku.

Dla każdego bloku 3x3 algorytm tworzy listę brakujących cyfr (blocks_missing) oraz listę współrzędnych pustych pól (empty_indices), które muszą zostać wypełnione tymi cyframi. Suma długości wszystkich list brakujących cyfr określa wymiarowość przestrzeni poszukiwań PSO (total_dimensions), która jest zazwyczaj znacznie mniejsza niż 81, szczególnie dla sudoku z większą liczbą początkowo uzupełnionych pól. Na przykład, jeśli sudoku ma 30 uzupełnionych pól, to pozostaje jedynie 51 pól do wypełnienia, ale niektóre bloki mogą już zawierać 6 lub 7 cyfr, co oznacza, że dla tych bloków musimy jedynie ustalić permutację 2-3 pozostałych cyfr.

```

def __init__(self, initial_sudoku):
    self.initial_grid = initial_sudoku.copy()
    self.blocks_missing = [] # Brakujące cyfry dla każdego bloku 3x3
    self.empty_indices = [] # Pozycje (r, c) pustych pól dla każdego bloku

    # Analiza struktury 3x3
    for r_offset in range(0, 9, 3):
        for c_offset in range(0, 9, 3):
            block = self.initial_grid[r_offset:r_offset+3,
                                      c_offset:c_offset+3]

            # Znajdź brakujące cyfry w bloku
            missing = [n for n in range(1, 10) if n not in block]
            self.blocks_missing.append(missing)

            # Znajdź indeksy tych brakujących cyfr
            empties = []
            for i in range(3):
                for j in range(3):
                    if block[i, j] == 0:
                        empties.append((r_offset + i, c_offset + j))
            self.empty_indices.append(empties)

    self.total_dimensions = sum(len(b) for b in self.blocks_missing)

```

5.2 Rekonstrukcja siatki

Funkcja `reconstruct_grid` stanowi serce implementacji i zawiera kluczową innowację - wykorzystanie sortowania do przekształcania ciągłych wartości PSO na dyskretne permutacje. Pozycja każdej cząstki w PSO to wektor liczb rzeczywistych (typowo w zakresie 0-1), ale sudoku wymaga dyskretnych cyfr 1-9. Tradycyjne podejścia często używają zaokrągleń lub mapowań modulo, co może prowadzić do duplikatów cyfr w blokach. Ta implementacja rozwiązuje ten problem elegancko - dla każdego bloku 3x3 pobiera fragment wektora pozycji cząstki odpowiadający temu blokowi, następnie używa funkcji `argsort` do uzyskania rankingów tych wartości. Jeśli na przykład blok potrzebuje cyfr [2, 5, 8] i fragment pozycji cząstki to [0.7, 0.3, 0.9], to `argsort` zwróci [1, 0, 2] (indeks najmniejszej wartości, środkowej, największej), co oznacza permutację: na pierwszą pozycję trafia druga cyfra z listy (5), na drugą pozycję pierwsza cyfra (2), na trzecią pozycję trzecia cyfra (8).

W ten sposób zawsze otrzymujemy wszystkie brakujące cyfry dokładnie raz w każdym bloku 3x3, bez możliwości duplikatów. Ten mechanizm automatycznie zapewnia, że jeden z trzech warunków sudoku (unikalność w blokach 3x3) jest zawsze spełniony, niezależnie od jakości rozwiązania, co znacząco upraszcza funkcję fitness i przyspiesza zbieżność algorytmu.

```

def reconstruct_grid(self, particle_pos):
    """
    Przekształca wektor ciągły PSO na poprawną planszę Sudoku.
    Wykorzystuje sortowanie (argsort) do stworzenia permutacji.
    """
    grid = self.initial_grid.copy()
    cursor = 0

    for block_idx, missing_vals in enumerate(self.blocks_missing):
        n_missing = len(missing_vals)
        if n_missing == 0:
            continue

        # Wycinamy fragment pozycji odpowiadający temu blokowi
        block_pos = particle_pos[cursor : cursor + n_missing]

        # Klucz do sukcesu: sortujemy brakujące wartości
        # według wag z PSO. Dzięki temu zawsze mamy
        # unikalne cyfry w bloku 3x3
        ranking = np.argsort(block_pos)
        for i, (r, c) in enumerate(self.empty_indices[block_idx]):
            grid[r, c] = missing_vals[ranking[i]]

        cursor += n_missing
    return grid

```

5.3 Funkcja fitness

Funkcja fitness ocenia jakość każdego rozwiązania reprezentowanego przez cząstkę w roju, koncentrując się wyłącznie na wierszach i kolumnach, ponieważ bloki 3x3 są zawsze poprawne dzięki mechanizmowi rekonstrukcji.

Dla każdej cząstki w roju funkcja najpierw rekonstruuje pełną siatkę sudoku z jej pozycji, następnie systematycznie sprawdza wszystkie 9 wierszy i 9 kolumn. Dla każdego wiersza oblicza liczbę unikalnych cyfr - w idealnym przypadku powinno być ich 9, co oznacza brak duplikatów. Różnica między ideałem a rzeczywistością ($9 - \text{unikalne}$) informuje, ile cyfr się powtarza w danym wierszu. Ta różnica jest podnoszona do kwadratu, tworząc nieliniową karę, która znacząco mocniej penalizuje wiersze z wieloma duplikatami niż te z pojedynczymi powtórzeniami. Identyfikacyjny proces wykonywany jest dla kolumn. Na przykład, jeśli wiersz zawiera cyfry $[1, 2, 3, 4, 5, 6, 7, 8, 8]$, to ma tylko 8 unikalnych cyfr, co daje karę $(9 - 8)^2 = 1$, ale wiersz z $[1, 1, 2, 2, 3, 3, 4, 4, 5]$ ma tylko 5 unikalnych cyfr, co daje znacznie większą karę $(9 - 5)^2 = 16$. Suma wszystkich tych kwadratowych kar z wszystkich wierszy i kolumn stanowi końcowy wynik fitness dla danej cząstki.

Algorytm PSO minimalizuje tę wartość, dążąc do stanu, gdzie $\text{fitness} = 0$, co odpowiada kompletnie rozwiązанemu sudoku z wszystkimi unikalnymi cyframi w każdym wierszu, kolumnie i bloku.

```

def fitness_func(self, swarm_pos):
    """
    Funkcja celu: minimalizujemy sumę kwadratów brakujących
    unikalnych cyfr w wierszach i kolumnach
    (bloki 3x3 są zawsze poprawne).
    """
    n_particles = swarm_pos.shape[0]
    penalties = np.zeros(n_particles)

    for i in range(n_particles):
        grid = self.reconstruct_grid(swarm_pos[i])
        score = 0

        # Sprawdzanie wierszy i kolumn
        for idx in range(9):
            # Kara za duplikaty w wierszu
            row_err = 9 - len(np.unique(grid[idx, :]))
            score += row_err**2

            # Kara za duplikaty w kolumnie
            col_err = 9 - len(np.unique(grid[:, idx]))
            score += col_err**2

        penalties[i] = score
    return penalties

```

5.4 Konfiguracja i uruchomienie algorytmu

Metoda solve konfiguruje i uruchamia algorytm PSO z starannie dobranymi parametrami, które kontrolują zachowanie cząstek w roju. Parametr $c1=1.4$ (współczynnik kognitywny) określa siłę przyciągania każdej cząstki do jej własnej najlepszej znalezionej pozycji - wyższa wartość 1.4 oznacza, że cząstki mają silną tendencję do eksploracji regionów, które wcześniej dla nich dobrze działały, co promuje indywidualne poszukiwania i może pomóc w unikaniu przedwczesnej zbieżności całego roju do jednego rozwiązania.

Parametr $c2=0.7$ (współczynnik społeczny) kontroluje siłę przyciągania do najlepszego globalnego rozwiązania znalezione przez cały rój - niższa wartość 0.7 (w porównaniu do $c1$) oznacza, że cząstki nie ślepo podążają za najlepszym rozwiązaniem roju, ale zachowują pewną niezależność, co zwiększa różnorodność eksploracji.

Parametr $w=0.8$ (bezwładność) określa, w jakim stopniu cząstka kontynuuje swój obecny kierunek ruchu - wartość 0.8 oznacza silną bezwładność, co powoduje, że cząstki nie zmieniają drastycznie kierunku w każdej iteracji, lecz poruszają się płynnie, co może pomóc w dokładniejszej eksploracji obiecujących regionów przestrzeni. Granice przestrzeni poszukiwań są ustawione na $[0,1]$ dla każdego wymiaru, ale konkretne wartości nie mają znaczenia - liczy się jedynie ich względna kolejność (ranking), która po zastosowaniu argsort determinuje permutację cyfr. Algorytm wykorzystuje wariant GlobalBestPSO, gdzie wszystkie cząstki w roju są świadome najlepszego globalnego rozwiązania i mogą do niego dążyć, co różni się od wariantów lokalnych, gdzie cząstki znają tylko rozwiązania w swoim sąsiedztwie.

```

def solve(self, n_particles=100, iters=1500):
    # Parametry PSO: c1-osobiste, c2-społeczne, w-bezwładność
    # Nieco wyższe c1 pozwala cząsteczkom bardziej
    # eksplorować własne ścieżki
    options = {'c1': 1.4, 'c2': 0.7, 'w': 0.8}

    # Granice: 0-1 (wartości nie mają znaczenia,
    # liczy się ich relacja/ranking)
    bounds = (np.zeros(self.total_dimensions),
              np.ones(self.total_dimensions))

    optimizer = ps.single.GlobalBestPSO(
        n_particles=n_particles,
        dimensions=self.total_dimensions,
        options=options,
        bounds=bounds
    )

    # Optymalizacja
    best_cost, best_pos = optimizer.optimize(
        self.fitness_func, iters=iters)

    return self.reconstruct_grid(best_pos), best_cost

```

5.5 Wyniki

Wynik algorytmu dla prostego problemu (40/81 początkowo zapełnionych pól) daje 100% poprawnych odpowiedzi, co pokazane jest na rysunku 10.

Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 36.6569 s.

Initial Sudoku Puzzle									Computed Solution								
9		2		7	8	4			9	6	2	3	7	8	4	1	5
1	8	5				7	6		1	8	5	4	2	9	7	6	3
	7		5						3	7	4	5	6	1	9	2	8
7	5				6		8	4	7	5	3	1	9	6	2	8	4
4		6				1	5	7	4	9	6	8	3	2	1	5	7
	1			4	5	3	9	6	2	1	8	7	4	5	3	9	6
	2					5		9	8	2	7	6	1	3	5	4	9
6		9			7	8	3		6	4	9	2	5	7	8	3	1
			9	8	4		7		5	3	1	9	8	4	6	7	2

Rysunek 10: Wynik działania algorytmu PSO dla prostego problemu.

Następnie sprawdzono działanie algorytmu na bardziej wymagającym problemie (38/81 początkowo zapełnionych pól). Wyniki widoczne są na rysunku 11. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 36,136 s. Algorytm osiągnął średni wynik na poziomie 95%

Initial Sudoku Puzzle									Computed Solution								
9	6					4			9	6	2	3	7	8	4	1	5
1	8	5	4	2					1	8	5	4	2	9	7	6	3
					1	9		8	3	7	4	5	6	1	9	2	8
5	3		9	8		6			5	3	1	9	8	4	6	3	2
	4	9						1	6	4	9	2	5	3	8	7	1
8	2	7					4	9	8	2	7	6	1	7	5	4	9
7	5		1		6		8		7	5	3	1	9	6	2	8	4
		6	8	3			5		4	9	6	8	3	2	1	5	7
	1	8	7			3		6	2	1	8	7	4	5	3	9	6

Rysunek 11: Wynik działania algorytmu PSO dla problemu średniej trudności.

Sprawdzono również działanie algorytmu na jeszcze bardziej wymagającym problemie (32/81 początkowo wypełnionych pól). Wyniki widoczne są na rysunku 12. Czas działania algorytmu jako średnia z 10 uruchomień wyniósł 42,151s. Algorytm osiągnął średni wynik na poziomie 82%

Initial Sudoku Puzzle									Computed Solution								
					1	2	3		6	9	7	9	5	1	2	3	8
1	2	3			8		4		1	2	3	6	4	8	9	4	7
8		4			7	6	5		8	5	4	2	3	7	6	5	1
7	6	5							7	6	5	7	2	3	4	8	9
									2	3	8	1	9	4	5	7	6
						1	2	3	9	4	1	8	6	5	1	2	3
	1	2	3			8		4	5	1	2	3	7	9	8	1	4
	8		4			7	6	5	3	8	9	4	8	2	7	6	5
	7	6	5						4	7	6	5	1	6	3	9	2

Rysunek 12: Wynik działania algorytmu PSO dla problemu ciężkiego.

6 Podsumowanie

Podsumowanie całej pracy można zaobserwować w tabeli 1.

Tabela 1: Porównanie wyników różnych algorytmów dla różnych poziomów trudności Sudoku

	Algorytm Backtracking	Algorytm Genetyczny	ACO	PSO
Problem prosty	100%(0,020s)	100%(12,320s)	100%(36,889s)	100%(36.6569s)
Problem średniej trudności	100%(0,033s)	100%(5,041s)	100%(34,621)	95%(36,136s)
Problem trudny	100%(0,041s)	86,5%(84,250s)	89%(42,151s)	82%(42,151s)

- **Algorytm Backtracking**

- Jest bardzo skuteczny dla wszystkich testowanych problemów, gdzie osiąga 100% poprawnych rozwiązań.
- Czas wykonania rośnie wraz z trudnością problemu, ale pozostaje na stosunkowo niskim poziomie, nawet dla problemu trudnego.

- **Algorytm Genetyczny**

- Wykazuje wysoką skuteczność dla problemów prostych i średniej trudności, osiągając 100% poprawnych rozwiązań.
- Czas wykonania znacznie rośnie wraz z trudnością problemu. Dla problemu trudnego czas wykonania jest znacząco wyższy w porównaniu do algorytmu Backtracking.

- **ACO (Ant Colony Optimization)**

- Osiąga doskonałe wyniki dla problemów prostych i średniej trudności (100% poprawnych rozwiązań), jednak dla problemu trudnego skuteczność spada do 89%.

- Czas wykonania jest stosunkowo stabilny niezależnie od trudności problemu, co świadczy o przewidywalnej złożoności obliczeniowej algorytmu.
- Charakterystyczna cecha algorytmu to tendencja do utkania w okolicach 90-95% rozwiązania - mechanizm feromonów prowadzi do zbieżności w kierunku dobrych, ale nie optymalnych rozwiązań, a następnie algorytm ma trudności z wyjściem z tego lokalnego optimum i doprowadzeniem rozwiązania do pełnych 100%.

- **PSO (Particle Swarm Optimization)**

- Wykazuje najniższą skuteczność spośród wszystkich testowanych algorytmów - 100% dla problemu prostego, 95% dla średniej trudności i tylko 82% dla problemu trudnego.
- Czas wykonania pozostaje względnie stabilny (około 36-42 sekund), niezależnie od trudności problemu.
- Algorytm przejawia wyraźną tendencję do przedwczesnej zbieżności - rój cząstek szybko osiąga rozwiązania na poziomie 90-95% poprawności, po czym utyka w tym stanie, nie mogąc przezwyciężyć ostatnich naruszeń reguł sudoku. Jest to typowy problem algorytmów rojowych, gdzie utrata różnorodności w populacji uniemożliwia dalszą eksplorację i znalezienie pełnego rozwiązania.

Podsumowując, wybór odpowiedniego algorytmu do rozwiązywania Sudoku zależy od poziomu trudności problemu oraz wymagań co do czasu wykonania. Algorytmy deterministyczne jak Backtracking są szybkie i skuteczne dla problemu rozwiązywania Sudoku. Natomiast metaheurystyki takie jak Algorytm Genetyczny, PSO i ACO dobrze radzą sobie z problemami mniej złożonymi, jednak potrzebują dłuższego czasu obliczeń.